

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

### Experiments

**COURSE NAME:** Computer Networks

**COURSE CODE:** CSA07

---

#### COURSE OUTCOME COVERED

**CO3:** *Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions*

Assessment Tool Weightage: 40%

---

#### **Code of Conduct:**

*I \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

#### **Signature of the student**

#### **Experiments**

#### **QUESTIONS:4**

**Total Marks:40**

1. Capture packets during a TCP connection setup. Identify the three packets involved in the handshake (SYN, SYN-ACK, ACK). Demonstrate the role of each flag and how they establish a reliable connection. (BL4) (10)
2. Capture a DNS query using UDP in Wireshark. Compare the UDP header size with TCP and note the absence of sequence numbers or acknowledgments. Discuss how this lightweight design impacts speed and reliability. (BL4) (10)
3. Simulate packet loss (e.g., disconnect briefly) and capture traffic in Wireshark. Observe how TCP retransmits lost packets while UDP does not. Explain why this difference makes TCP reliable but slower compared to UDP. (BL4) (10)
4. Use Wireshark to capture DNS traffic while resolving a domain name. Identify the query type (A, AAAA, MX) and the response received. Measure the time taken for resolution and explain why DNS typically uses UDP.(BL4) (10)

#### **RUBRICS FOR CALCULATION:**

| Criteria | Weightage | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|----------|-----------|---------------------|----------------------|----------------------|---------------------|
|----------|-----------|---------------------|----------------------|----------------------|---------------------|

|                           |    |                                                                                               |                                                             |                                                     |                                                  |
|---------------------------|----|-----------------------------------------------------------------------------------------------|-------------------------------------------------------------|-----------------------------------------------------|--------------------------------------------------|
| Understanding of Concepts | 25 | Demonstrates thorough understanding and effectively integrates multiple concepts/technologies | Good understanding with proper integration of most concepts | Basic understanding; limited or partial integration | Poor understanding; unable to integrate concepts |
| Experimental Design       | 30 | Well-planned, systematic implementation with optimal solution                                 | Correct implementation with minor issues                    | Basic implementation with errors or inefficiencies  | Incorrect or incomplete implementation           |
| Use of Tools              | 20 | Effective and advanced use of tools/technologies with proper configuration                    | Appropriate use of tools with correct execution             | Limited or inconsistent use of tools                | Incorrect or minimal use of tools                |
| Analysis of Results       | 15 | Insightful analysis with accurate interpretation and validation of results                    | Good analysis with reasonable interpretation                | Basic analysis with limited insights                | Poor or incorrect analysis of results            |
| Documentation             | 10 | Clear, well-structured documentation with diagrams, code, and results                         | Organized documentation with necessary details              | Basic documentation with missing details            | Poor or incomplete documentation                 |

## Question 1 — TCP Three-Way Handshake (10 Marks)

*Capture packets during a TCP connection setup. Identify the three packets involved in the handshake (SYN, SYN-ACK, ACK). Demonstrate the role of each flag and how they establish a reliable connection.*

### 1.1 Objective

To use Wireshark to capture live traffic during the establishment of a TCP connection, identify the SYN, SYN-ACK and ACK segments that make up the three-way handshake, and explain how the flags and sequence/acknowledgment numbers in each segment cooperate to establish a reliable, connection-oriented session between a client and a server.

### 1.2 Tools / Software Required

- Wireshark (packet capture and analysis)
- A host machine with an active network interface (Wi-Fi/Ethernet)
- A web browser or terminal utility (curl / telnet) to generate a fresh TCP connection, e.g. to a website on port 80/443

### 1.3 Procedure

1. Open Wireshark and select the active network interface (e.g. Wi-Fi or Ethernet).
2. Start a capture, and apply the display filter `tcp.flags.syn == 1 || (ip.addr == <server_ip> && tcp)` to isolate the connection of interest.
3. In a browser or terminal, initiate a new connection to a web server that is not already cached/connected, e.g. open `http://example.com` or run `curl http://example.com`.
4. Stop the capture and locate the first three packets exchanged between the client and the server for that connection.

- Click each packet and expand the 'Transmission Control Protocol' section in the packet-details pane to inspect the Flags, Sequence Number and Acknowledgment Number fields.
- Right-click the first SYN packet and choose Follow → TCP Stream to see the whole conversation isolated from other traffic.

## 1.4 Sample Captured Output

A representative capture of a handshake between a client (192.168.1.10) and a web server (93.184.216.34) is summarised below. Replace this with the values from your own capture.

| No. | Time     | Source        | Destination   | Flags       | Seq / Ack      | Info                                                                              |
|-----|----------|---------------|---------------|-------------|----------------|-----------------------------------------------------------------------------------|
| 1   | 0.000000 | 192.168.1.10  | 93.184.216.34 | SYN         | Seq=0          | Client → Server:<br>49514 → 80 [SYN]<br>Seq=0 Win=64240<br>MSS=1460               |
| 2   | 0.028417 | 93.184.216.34 | 192.168.1.10  | SYN,<br>ACK | Seq=0<br>Ack=1 | Server → Client: 80<br>→ 49514 [SYN,<br>ACK] Seq=0 Ack=1<br>Win=65535<br>MSS=1460 |
| 3   | 0.028562 | 192.168.1.10  | 93.184.216.34 | ACK         | Seq=1<br>Ack=1 | Client → Server:<br>49514 → 80 [ACK]<br>Seq=1 Ack=1<br>Win=64240                  |

## Question 2 — DNS over UDP vs. TCP Header Comparison (10 Marks)

Capture a DNS query using UDP in Wireshark. Compare the UDP header size with TCP and note the absence of sequence numbers or acknowledgments. Discuss how this lightweight design impacts speed and reliability.

### 2.1 Objective

To capture a DNS lookup carried over UDP, examine the UDP header fields in Wireshark, contrast the header structure and size with a TCP header, and reason about the speed/reliability trade-off that results from UDP's minimal, stateless design.

### 2.2 Tools / Software Required

- Wireshark
- Command-line DNS tools: nslookup or dig (dig purge/+notcp to force UDP)

### 2.3 Procedure

- Start a Wireshark capture and apply the filter `udp.port == 53` (or simply `dns`).
- Flush the local DNS cache (`ipconfig /flushdns` on Windows, `sudo systemd-resolve --flush-caches` on Linux) so the query is not answered from cache.
- Run `nslookup www.example.com` (or `dig www.example.com`) from a terminal.
- In Wireshark, select the outgoing DNS query packet and expand 'User Datagram Protocol' in the packet-details pane to inspect the UDP header fields.

11. For comparison, select any TCP segment already captured (e.g. from Question 1) and expand 'Transmission Control Protocol' to view its header fields side-by-side.

## 2.4 Sample Captured Output

| No. | Time     | Source       | Destination  | Protocol | Length | Info                                                             |
|-----|----------|--------------|--------------|----------|--------|------------------------------------------------------------------|
| 1   | 0.000000 | 192.168.1.10 | 8.8.8.8      | DNS      | 70     | Standard query 0x1a2b A www.example.com                          |
| 2   | 0.021340 | 8.8.8.8      | 192.168.1.10 | DNS      | 86     | Standard query response 0x1a2b A www.example.com A 93.184.216.34 |

## Question 3 — Packet Loss: TCP Retransmission vs. UDP (10 Marks)

*Simulate packet loss (e.g., disconnect briefly) and capture traffic in Wireshark. Observe how TCP retransmits lost packets while UDP does not. Explain why this difference makes TCP reliable but slower compared to UDP.*

### 3.1 Objective

To induce packet loss during an active TCP file/data transfer and during a UDP stream, capture both in Wireshark, and compare how each protocol behaves — specifically, to observe TCP's retransmission mechanism and confirm that UDP performs no such recovery.

### 3.2 Tools / Software Required

- Wireshark
- A sustained TCP transfer to observe (e.g. downloading a large file with curl/wget, or streaming a video)
- A UDP stream to observe (e.g. an online UDP-based speed test, a VoIP/video call, or iperf3 -u between two hosts)
- A way to induce brief loss: toggling Wi-Fi off/on for 1-2 seconds, temporarily unplugging Ethernet, or using a network-conditioning tool (e.g. Clumsy on Windows, tc netem on Linux) to drop a percentage of packets

### 3.3 Procedure

12. Start a Wireshark capture on the active interface.
13. Begin a large TCP download (e.g. `curl -O <large-file-url>`) or start a video call/stream.
14. While the transfer is running, briefly disable and re-enable the network adapter (or apply artificial packet loss with a conditioning tool) for 1–2 seconds, then let the transfer continue.
15. Repeat the same brief-disconnect test while a UDP stream (e.g. iperf3 -u or a live video call) is running, in a separate capture.
16. Stop each capture and apply the filter `tcp.analysis.retransmission` on the TCP capture to isolate retransmitted segments, and inspect the UDP capture for any recovery packets (there will be none).
17. Use Statistics → I/O Graph in Wireshark to visualise the drop in throughput and the recovery period for both captures.

### 3.4 Sample Captured Output — TCP (retransmission visible)

| No. | Time  | Source → Dest                | Info                                       |
|-----|-------|------------------------------|--------------------------------------------|
| 145 | 5.201 | 192.168.1.10 → 93.184.216.34 | [TCP Retransmission] Seq=48001<br>Len=1460 |
| 146 | 5.203 | 192.168.1.10 → 93.184.216.34 | [TCP Retransmission] Seq=49461<br>Len=1460 |
| 150 | 5.640 | 93.184.216.34 → 192.168.1.10 | [TCP Dup ACK] Ack=48001                    |
| 162 | 6.812 | 93.184.216.34 → 192.168.1.10 | ACK Ack=52381 (transfer resumes normally)  |

## Question 4 — DNS Query Types, Response, and Resolution Time (10 Marks)

Use Wireshark to capture DNS traffic while resolving a domain name. Identify the query type (A, AAAA, MX) and the response received. Measure the time taken for resolution and explain why DNS typically uses UDP.

### 4.1 Objective

To capture the DNS traffic generated while resolving a domain name, identify the resource-record query type(s) used and the answers returned, measure the query-to-response resolution time from the Wireshark timestamps, and explain the rationale for DNS's default use of UDP.

### 4.2 Tools / Software Required

- Wireshark
- dig (supports explicit record-type queries, e.g. dig example.com A , dig example.com AAAA , dig example.com MX )

### 4.3 Procedure

- 18.Start a Wireshark capture with the filter dns applied.
- 19.Flush the local DNS cache to force a fresh lookup.
- 20.Run three separate lookups to generate different query types: dig example.com A , dig example.com AAAA , and dig example.com MX .
- 21.Stop the capture. For each query, select the request packet and note its 'Time' column value, then select the matching response packet (same Transaction ID) and note its 'Time' value.
- 22.Expand each DNS layer in the packet-details pane to read the Queries and Answers sections, confirming the Type field (A / AAAA / MX) and the returned record.
- 23.Compute the resolution time for each query as (response timestamp – query timestamp).

### 4.4 Sample Captured Output

| No. | Time (s) | Source → Dest          | Query Type | Info                                |
|-----|----------|------------------------|------------|-------------------------------------|
| 1   | 0.000000 | 192.168.1.10 → 8.8.8.8 | A          | Standard query 0x5c11 A example.com |

|   |          |                           |      |                                                                                               |
|---|----------|---------------------------|------|-----------------------------------------------------------------------------------------------|
| 2 | 0.024118 | 8.8.8.8 →<br>192.168.1.10 | A    | Standard query response 0x5c11 A<br>example.com A 93.184.216.34                               |
| 3 | 1.201355 | 192.168.1.10 →<br>8.8.8.8 | AAAA | Standard query 0x7d2a AAAA<br>example.com                                                     |
| 4 | 1.229840 | 8.8.8.8 →<br>192.168.1.10 | AAAA | Standard query response 0x7d2a AAAA<br>example.com AAAA<br>2606:2800:220:1:248:1893:25c8:1946 |
| 5 | 2.503010 | 192.168.1.10 →<br>8.8.8.8 | MX   | Standard query 0x9e44 MX example.com                                                          |
| 6 | 2.541677 | 8.8.8.8 →<br>192.168.1.10 | MX   | Standard query response 0x9e44 MX<br>example.com 0 . (no mail exchanger<br>configured)        |